



VM: III

Types of Hypervisors

- **Type 0 Hypervisors:**

- Hardware/Firmware-based virtualization.
- Found in mainframes & large servers.
- Examples: **IBM LPARs, Oracle LDOMs.**

- **Type 1 Hypervisors (Bare-metal):**

- OS-like virtualization software running directly on hardware.
- Examples: **VMware ESX, Citrix XenServer, Joyent SmartOS.**
- Also includes OSs with built-in VMMs: **Windows Server (Hyper-V), Red Hat KVM.**

- **Type 2 Hypervisors (Hosted):**

- Run as applications on existing OS.
- Examples: **VMware Workstation, Parallels Desktop, Oracle VirtualBox.**

Other Virtualization Techniques

- **Paravirtualization:**

- Guest OS modified to work with VMM directly.
- Improves speed and reduces overhead.
- *Example:* Xen hypervisor.

- **Programming Environment Virtualization:**

- Virtualizes software platforms, not hardware.
- Runs apps in a consistent runtime environment.
- *Examples:* Java (JVM), .NET (CLR).

- **Emulation:**

- Imitates different hardware so apps can run on other systems.
- *Example:* QEMU runs ARM apps on x86 machines.

- **Application Containment:**

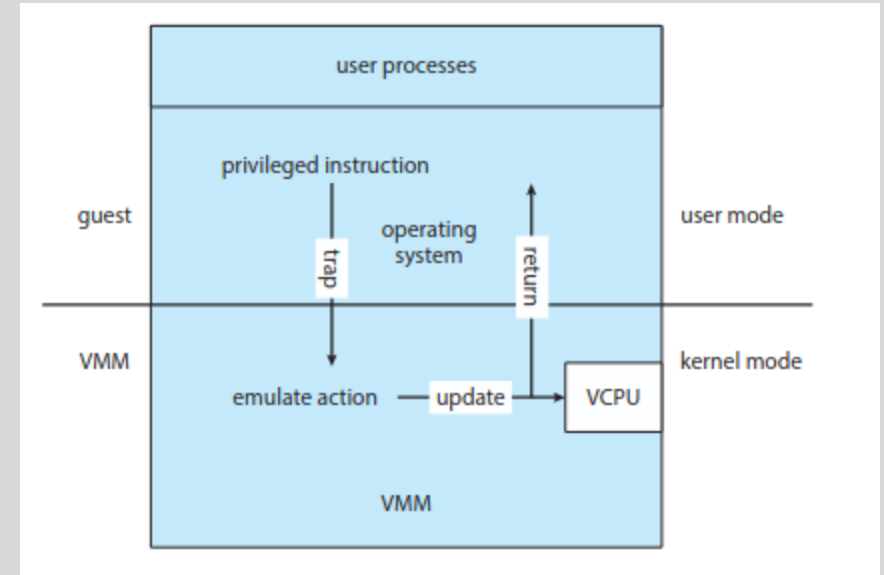
- Isolates apps for security and manageability.
- Not full virtualization.
- *Examples:* Solaris Zones, BSD Jails, AIX WPARs.

Building Blocks of Virtualization

- Hard to make an exact copy of the real machine
- More difficult on systems with only user and kernel modes
- CPU features decide if virtualization is possible
- VMM (Virtual Machine Monitor) uses:
 - Trap-and-emulate
 - Binary translation
- Virtual CPU (VCPU):
 - Shows what the guest thinks its CPU looks like
 - Doesn't run code, just stores CPU state
 - Used when switching between guests (like PCB in OS)
- Type 0 hypervisors don't need these building blocks

Trap-and-Emulate Virtualization

- Guest OS runs in **user mode** (can't access kernel mode directly).
- VMM (Virtual Machine Monitor) manages privileged operations safely.
- Two virtual modes are created: **virtual user mode** and **virtual kernel mode**.
- Working
 1. Guest OS tries to execute a **privileged instruction**.
 2. This causes a **trap** to the VMM (since it's not allowed in user mode).
 3. VMM **emulates** the instruction on behalf of the guest.
 4. VMM **updates** the VCPU state.
 5. Control is **returned** to the guest OS.



Performance

- **Nonprivileged instructions** → run directly on hardware (fast).
- **Privileged instructions** → emulated (slower, adds overhead).
- VMM must share CPU among multiple VMs → can cause slowdowns.
- Some CPUs handle guest CPU state in hardware, so the VMM doesn't need to manage it, cutting down virtualization overhead.

Binary Translation

- **Trap-and-emulate** relies on privileged instructions causing a **trap (an interrupt)** when run in user mode, so the VMM can catch and safely handle them.
- Some instructions of x86(e.g., `popf`) don't trap in user mode, making trap-and-emulate ineffective.
- Binary translation handles this by:
 - **User mode:** Guest runs instructions directly on CPU (natively).
 - **Kernel mode:** Guest thinks it's running in kernel mode, but:
 - VMM reads upcoming instructions before execution.
 - Normal instructions run directly.
 - Special instructions are translated into safe, equivalent instructions.

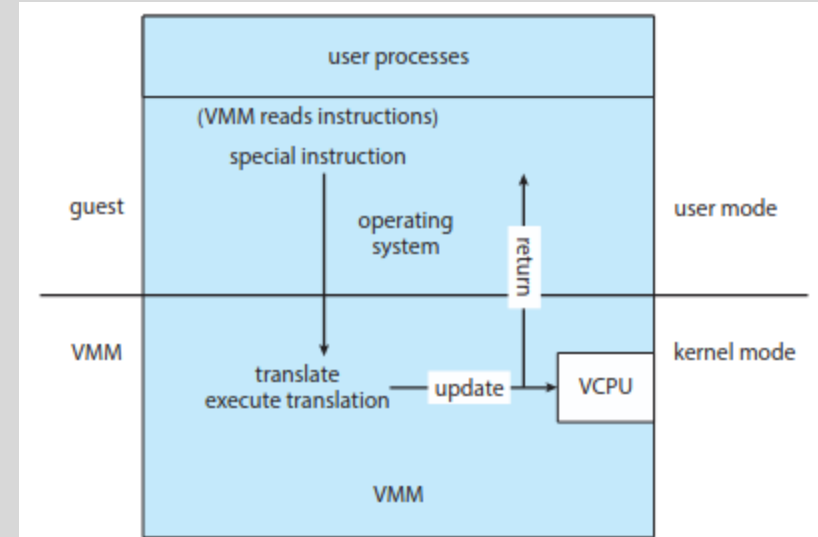


Figure 18.3 Binary translation virtualization implementation.

Hardware Assistance in Virtualization

- Virtualization relies heavily on hardware support.
- Intel introduced VT-x in 2005; AMD introduced AMD-V in 2006.
- These add host mode and guest mode to CPUs (moving from dual-mode to multi-mode).
- Guest OS runs in guest mode, thinking it controls hardware directly.
- VMM manages transitions between host and guest modes.

Hardware Support for Memory Management

- AMD's (Rapid Virtualization Indexing) **RVI** and Intel's (Extended Page table) **EPT** implement nested page tables (NPT) in hardware.
- NPTs add a layer for guest-to-host memory translation, improving efficiency.
- CPU walks through guest page table and VMM page table to resolve addresses.
- Extra translation can cause more TLB misses, affecting performance.

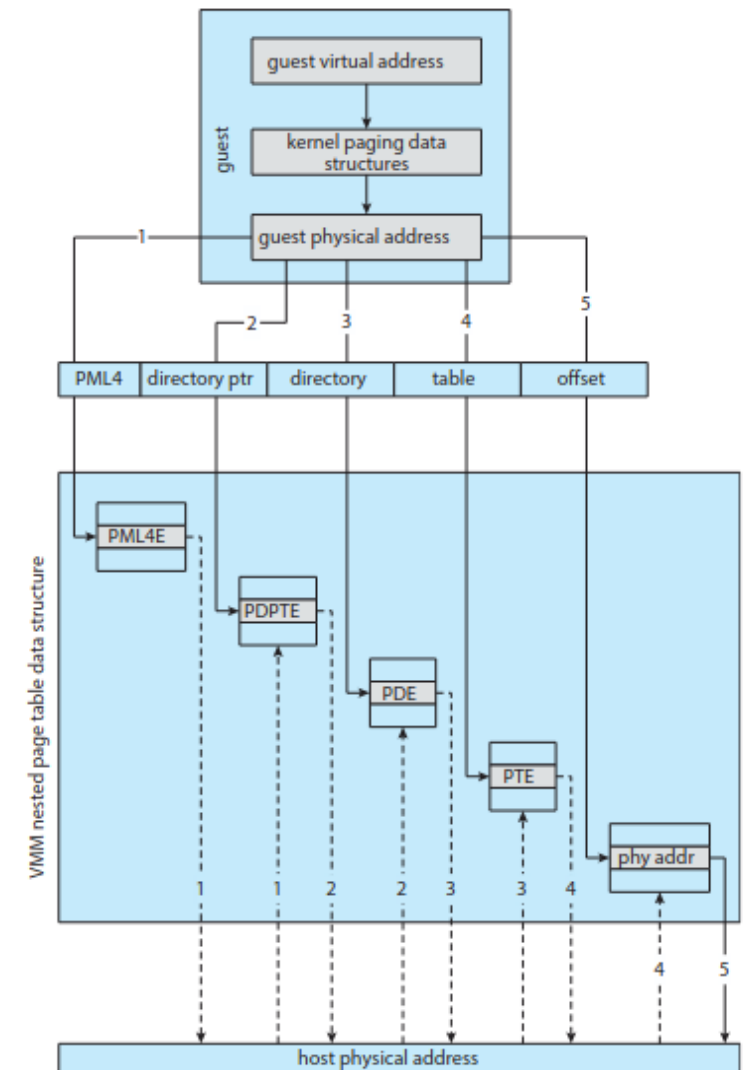


Figure 18.4 Nested page tables.

Nested Page Table

- **Guest Virtual Address (GVA):** The address used by the Guest OS (the VM) to access memory.
- **Guest Physical Address (GPA):** The address the Guest OS thinks it is accessing.
- **Host Physical Address (HPA):** The actual physical address on the Host OS's physical memory (the memory managed by the hypervisor).

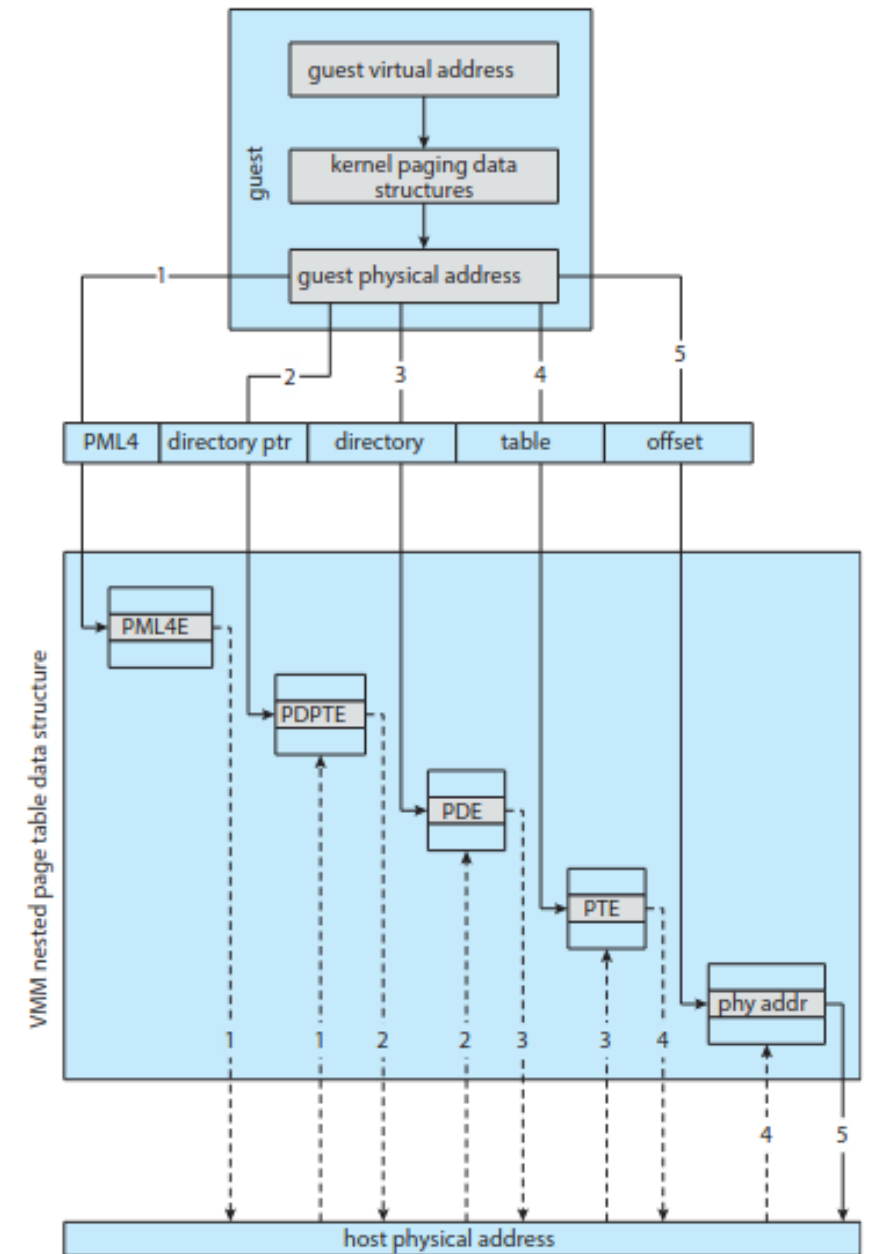


Figure 18.4 Nested page tables.

Nested Page Table

- **PML4E (Page Map Level 4 Entry)**-The PML4 contains pointers to **PDPTEs** (Page Directory Pointer Tables), which in turn point to PDPTE.
- **PDPTE (Page Directory Pointer Table Entry)**-A **PDPTE** is a pointer to a **Page Directory**, which helps in managing larger sections of memory
- **PDE (Page Directory Entry)**- A **PDE** points to an entry in the **Page Table** (PTE).
- **PTE (Page Table Entry)**-PTEs point to the physical memory addresses or the actual pages in physical memory.

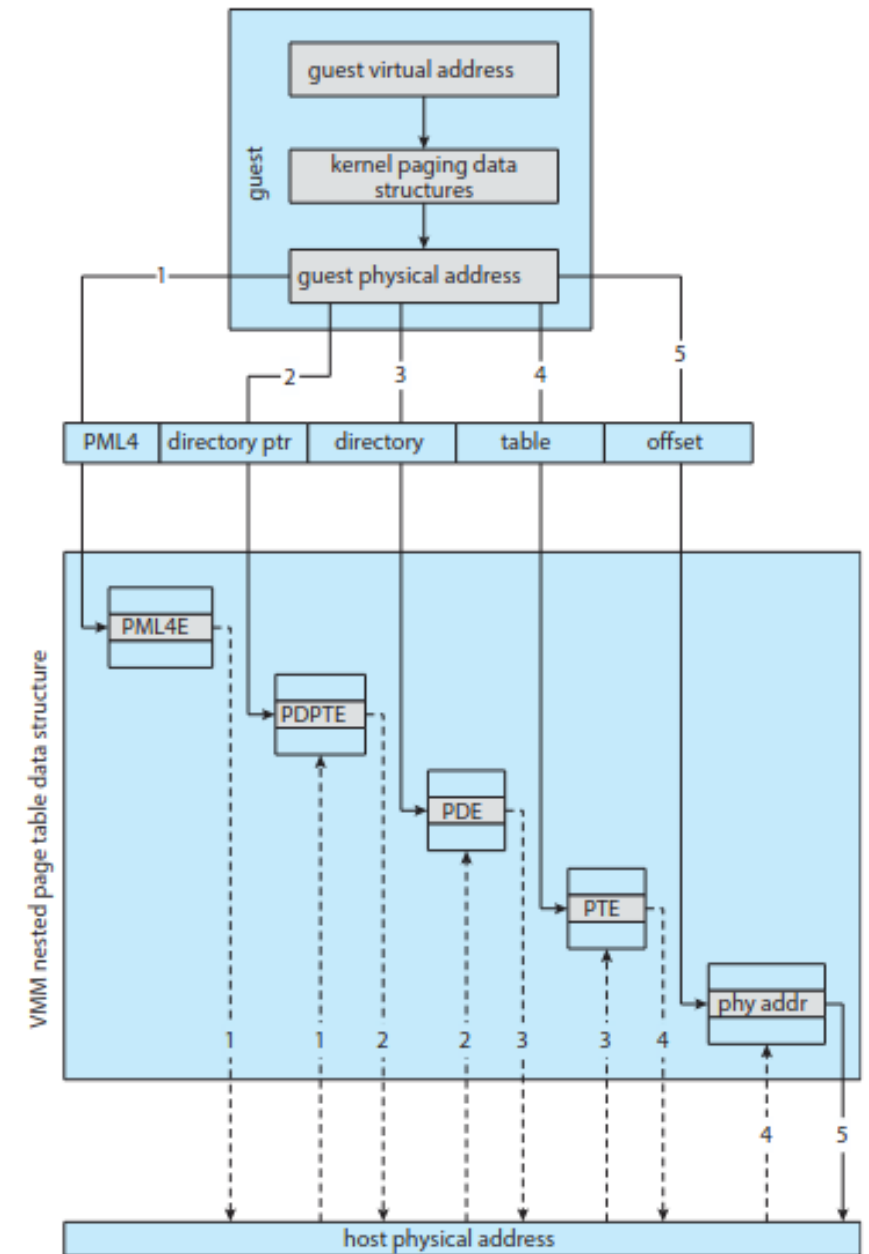


Figure 18.4 Nested page tables.

Step	Process	Example
1. Guest Virtual Address (GVA)	Guest OS wants to access memory.	GVA = 0x0000000000012345
2. Guest Page Tables	Guest OS translates GVA to GPA .	GPA = 0x0000ABCDE1234567
3. Nested Page Tables (NPT)	VMM uses NPT to map GPA to HPA .	HPA = 0x000000001234ABCD
4. Physical Memory Access	The actual memory access happens at HPA .	Access physical memory at HPA

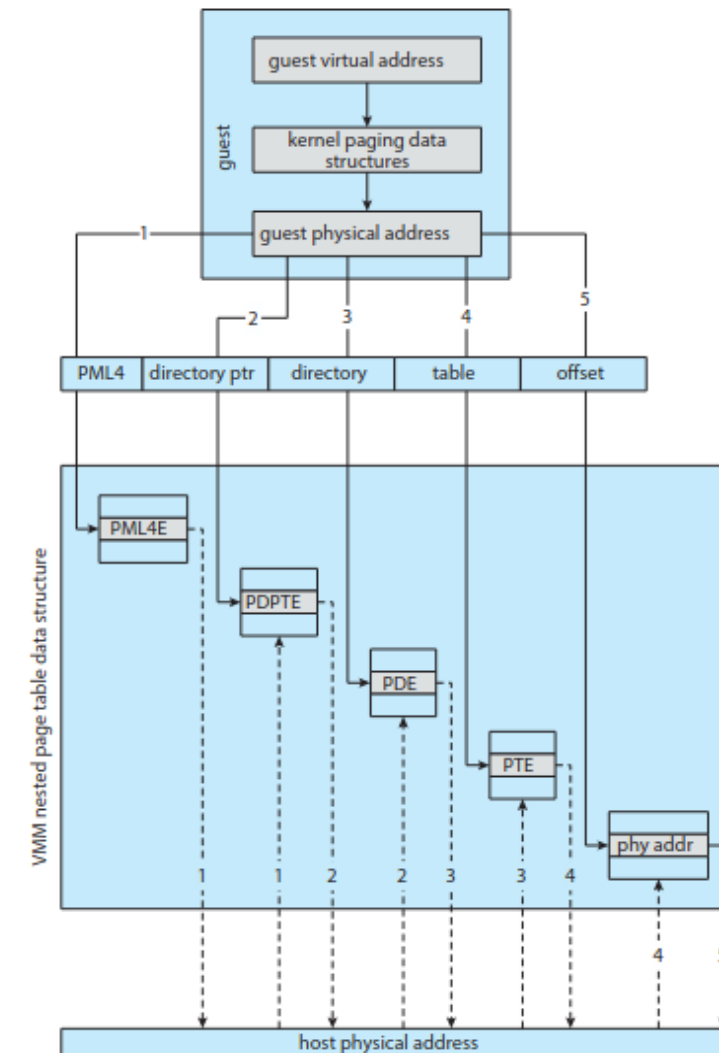


Figure 18.4 Nested page tables.

Hardware-Assisted I/O and Interrupts

- DMA controllers now use hardware assistance to prevent guest memory access violations.
- VMM sets up **protection domains** to control device access.
- CPUs translate DMA addresses safely without VMM interference.
- Interrupt remapping directs interrupts to the correct guest, improving security and efficiency.

CPU Scheduling in Virtualized Systems

- **Virtual CPUs and Physical CPUs**
 - Virtual machines (VMs) get one or more **virtual CPUs (VCPUs)**.
 - VMM schedules physical CPUs among guest VCPUs.
- **CPU Allocation Scenarios**
 - **Enough physical CPUs:** VCPUs can be dedicated; guests run like native OS.
 - **Overcommitment:** More VCPUs than physical CPUs; VMM shares CPU time fairly among guests.
- **Scheduling**
 - Guest OS scheduling assumes fixed CPU time slices.
 - Virtualized environments cause CPU time slices to **take longer** than expected (e.g., 100 ms slice may take seconds).
 - Leads to poor user responsiveness, especially in time-sharing or real-time systems.
- **Effects of Virtualization**
 - Guests receive only a fraction of physical CPU cycles despite scheduling full CPU time internally.
 - Virtual machine clocks may drift or run inaccurately.
- **Solutions**
 - VMM provides **guest tools/apps** to correct clock drift and improve device management inside VMs.

Virtualization and I/O

- **Virtualized Devices**

- VMM provides virtual devices to guest OS.
- Guest OS sees a simple, idealized device; VMM handles communication with real hardware.

- **Direct Device Access**

- Type 0, 1, 2 hypervisors can allow guests to access hardware directly.
- Improves I/O performance, can be near-native speed.
- Requires hardware support like DMA pass-through and direct interrupt delivery (VT-d).

- **Shared Devices**

- VMM manages access to shared devices (e.g., disk drives).
- Ensures protection and correctness for each guest.

- **Networking**

- Each guest needs its own IP address.
- VMM acts as a **virtual switch** for routing.
- Options:
 - **Bridging:** Guest has external IP.
 - **NAT:** VMM provides local IP, manages routing.
- VMM can also provide **firewalling** between guests and external network.

Storage Management in Virtualization

- **Boot Disk Challenge:** Standard partitioning insufficient for many VMs.
- **Hypervisor Solutions:**
 - **Type 0:** Uses root disk partitions or a disk manager to give each guest its storage.
 - **Type 1:** Keeps each guest's disk as a file inside the hypervisor.
 - **Type 2:** Stores each guest's disk as a file on the host operating system.
- **Disk Image Benefits:** Easy copying, moving, duplicating VMs.
- **Expanding Storage:** VMM creates additional virtual disks; guest disk I/O mapped to host file I/O.
- **Physical ↔ Virtual Conversion:**
 - P-to-V: Capture physical system into a VM.
 - V-to-P: Convert VM back to physical system for testing/debugging.

Live Migration

- Running guest can be moved between systems, without interrupting user access to the guest or its apps
- Very useful for resource management, maintenance downtime, etc.
 1. The source VMM establishes a connection with the target VMM.
 2. The target creates a new guest by creating a new VCPU, etc.
 3. The source sends all read-only guest memory pages to the target.
 4. The source sends all read-write pages to the target, marking them clean.
 5. The source repeats step 4, as during that step some pages were probably modified by the guest and are now dirty.
 6. Source VMM sends VCPU's final state details, sends final dirty pages, and tells target to start running the guest.
 7. Once target acknowledges that guest running, source terminates guest.

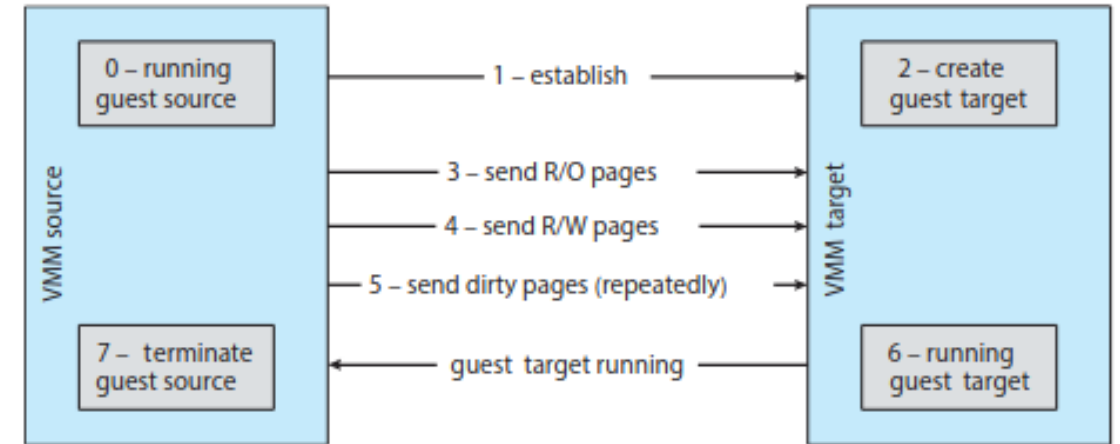


Figure 18.8 Live migration of a guest between two servers.

Paravirtualization via Xen

- Xen: most popular example of paravirtualized VMM
- Paravirtualization: modify guest OS to be amenable to virtualization
 - XenLinux is a modified Linux OS that runs on Xen hypervisor
 - Application interface need not change
- Benefits: better performance than binary translation
- Disadvantages: requires source code changes to OS, porting effort
- 1-2% code changes reported in the original Xen paper

Xen Architecture

- Type 1 hypervisor: runs directly over hardware
- Trap-and-emulate architecture
 - Xen runs in ring 0, guest OS in ring 1
 - Xen sits in the top 64MB of address space of guests, so guest OS memory doesn't overlap with the hypervisor.
 - Guest OS traps to Xen to perform privileged actions
- A guest VM is called a domain
 - Special domain called dom0 runs control/management software

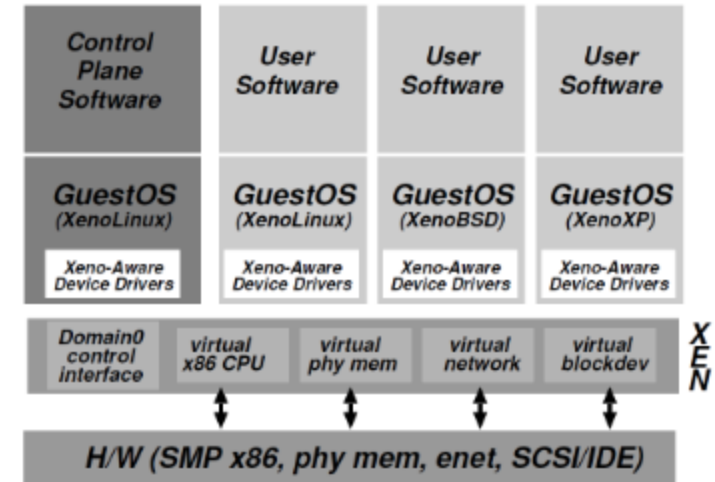


Figure 1: The structure of a machine running the Xen hypervisor, hosting a number of different guest operating systems, including *Domain0* running control software in a XenoLinux environment.

CPU virtualization in Xen

- Guest OS code modified to not invoke any privileged instruction
 - Any privileged operation traps to Xen in ring 0
- Hypercalls: guest OS voluntarily invokes Xen to perform privileged ops
 - Much like system calls from user process to kernel
 - Synchronous: guest pauses while Xen services the hypercall
- Asynchronous event mechanism: Communication from Xen to domain
 - Much like interrupts from hardware to kernel
 - Used to deliver hardware interrupts and other notifications to domain
 - Domain registers event handler callback functions

Trap handling in Xen

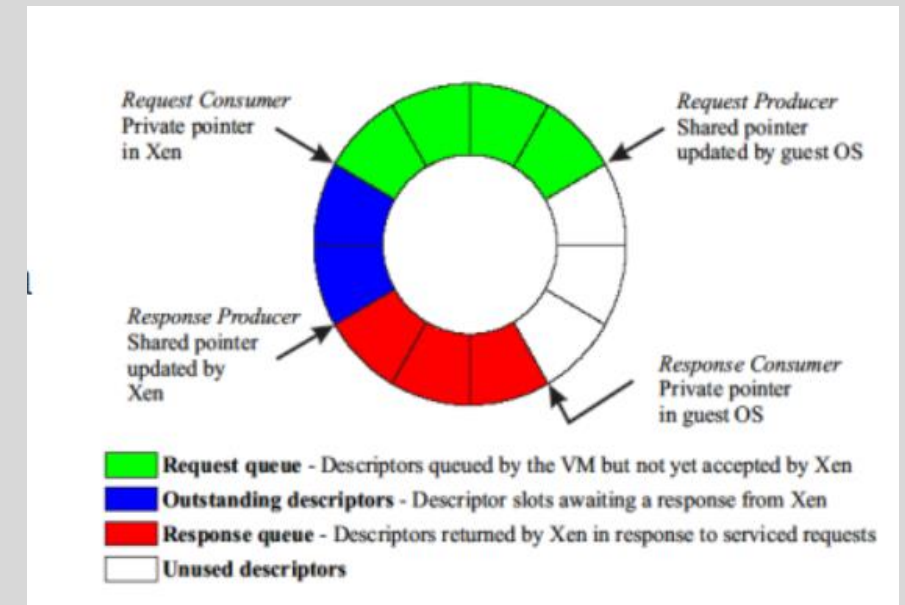
- When trap/interrupt occurs, Xen copies the trap frame onto the guest OS kernel stack, invokes guest interrupt handler
- Guest registers an interrupt descriptor table with Xen to handle traps
 - Interrupt handlers validated by Xen (check that no privileged segments loaded)
- Guest trap handlers work off information on kernel stack, no modifications needed to guest OS code
 - Except page fault handler, which needs to read CR2 register to find faulting address (privileged operation)
 - Page fault handler modified to read faulting address from kernel stack (address placed on stack by Xen)
- What if interrupt handler still invokes privileged operations?
 - Traps to Xen again and Xen detects this “double fault” (trap followed by another trap from interrupt handler code) and terminates misbehaving guest

Memory virtualization in Xen

- One copy of combined GVA -> HPA page table maintained by guest OS
 - CR3 points to this page table
 - Like shadow page tables, but in guest memory, not in VMM
- Guest is given read-only access to guest “RAM” mappings (GPA->HPA)
 - Using this, guest can construct combined GVA->GPA mapping
- Guest page table is in guest memory, but validated by Xen
 - Guest marks its page table pages as read-only, cannot modify
 - When guest needs to update, it makes a hypercall to Xen to update page table
 - Xen validates updates (is guest accessing its slice of RAM?) and applies them
 - Batched updates for better performance
- Segment descriptor tables are also maintained similarly
 - Read-only copy in guest memory, updates validated and applied by Xen
 - Segments truncated to exclude top 64MB occupied by Xen

I/O virtualization in Xen

- Shared memory “rings” between guest domain and Xen/domain0
 - Front-end device driver in guest domain and backend in dom0
 - I/O requests placed in shared queue by guest domain
 - Request handled by Xen/domain0, responses placed in ring
 - Descriptors in queue: pointers to request data (DMA buffers with data for writes, empty DMA buffers for reads, etc.)
- Similar design to virtio
 - No copying of request data
 - Memory pages swapped between domains to exchange requests/responses
 - Batching for high performance



Summary

- Xen: paravirtualization based hypervisor
- Guest OS modified to suit virtualization
- Trap-and-emulate via VMM
 - Guest in ring 1, VMM in ring 0
 - Guest traps to VMM for privileged operations
- Combined GVA
 - HPA page tables in guest memory
 - Read-only copy in guest
 - Updated via hypercalls to Xen
- I/O via shared rings between guest and Xen/domain0



Thank You